

hwk2/memory.vhd

```

1  -----
2  --
3  -- Memory Subsystem
4  --
5  -- This component describes the memory for a 32-bit byte-addressable CPU
6  -- with a 32-bit address bus. Only a portion of the full address space is
7  -- filled in. Addresses outside the filled in range return 'X' when read
8  -- and generate error messages when written.
9  --
10 -- Revision History:
11 --      28 Apr 25  Glen George      Initial revision.
12 --      29 Apr 25  Glen George      Fixed some syntax errors.
13 --      29 Apr 25  Glen George      Fixed inconsistencies in byte vs word
14 --                                  addressing.
15 --      30 Apr 26  Simone Shevchuk  Parens fixed, RAM_SIZE added
16 --      01 May 26  Simone Shevchuk  Change write end_if add wait for 0 ns after
17 --      02 May 26  Simone Shevchuk  Add comments
18  -----
19
20
21 --
22 -- MEMORY32x32
23 --
24 -- This is a memory component that supports a byte-addressable 32-bit wide
25 -- memory with 32-bits of address. No timing restrictions are implemented,
26 -- but if the address bus changes while a WE signal is active an error is
27 -- generated. Only a portion of the memory is actually usable. Addresses
28 -- outside of the four usable ranges return 'X' on read and generate error
29 -- messages on write. The size and address of each memory chunk are generic
30 -- parameters.
31 --
32 -- Generics:
33 --      MEMSIZE      - size of the four memory blocks in 32-bit words
34 --      START_ADDR0  - starting address of first memory block/chunk
35 --      START_ADDR1  - starting address of second memory block/chunk
36 --      START_ADDR2  - starting address of third memory block/chunk
37 --      START_ADDR3  - starting address of fourth memory block/chunk
38 --
39 -- Inputs:
40 --      RE0          - low byte read enable (active low)
41 --      RE1          - byte 1 read enable (active low)
42 --      RE2          - byte 2 read enable (active low)
43 --      RE3          - high byte read enable (active low)
44 --      WE0          - low byte write enable (active low)
45 --      WE1          - byte 1 write enable (active low)
46 --      WE2          - byte 2 write enable (active low)
47 --      WE3          - high byte write enable (active low)

```

```

48  --      MemAB  - memory address bus (32 bits)
49  --
50  --      Inputs/Outputs:
51  --      MemDB  - memory data bus (32 bits)
52  --
53
54  library ieee;
55  use ieee.std_logic_1164.all;
56  --use ieee.std_logic_arith.all;
57  --use ieee.std_logic_unsigned.all;
58  use ieee.numeric_std.all;
59
60
61  entity MEMORY32x32 is
62
63      generic (
64          MEMSIZE      : integer := 256;    -- default size is 256 words
65          START_ADDR0  : integer;           -- starting address of first block
66          START_ADDR1  : integer;           -- starting address of second block
67          START_ADDR2  : integer;           -- starting address of third block
68          START_ADDR3  : integer;           -- starting address of fourth block
69      );
70
71      port (
72          RE0   : in    std_logic;          -- low byte read enable (active low)
73          RE1   : in    std_logic;          -- byte 1 read enable (active low)
74          RE2   : in    std_logic;          -- byte 2 read enable (active low)
75          RE3   : in    std_logic;          -- high byte read enable (active low)
76          WE0   : in    std_logic;          -- low byte write enable (active low)
77          WE1   : in    std_logic;          -- byte 1 write enable (active low)
78          WE2   : in    std_logic;          -- byte 2 write enable (active low)
79          WE3   : in    std_logic;          -- high byte write enable (active low)
80          MemAB : in    std_logic_vector(31 downto 0); -- memory address bus
81          MemDB : inout std_logic_vector(31 downto 0) -- memory data bus
82      );
83
84  end MEMORY32x32;
85
86
87  architecture behavioral of MEMORY32x32 is
88      constant RAM_SIZE : integer := MEMSIZE; -- size of each memory block in bytes
89
90      -- define the type for the RAM chunks
91      type RAMtype is array (0 to MEMSIZE - 1) of std_logic_vector(31 downto 0);
92
93      -- now define the RAMs (initialized to X)
94      signal RAMbits0 : RAMtype := (others => (others => 'X'));
95      signal RAMbits1 : RAMtype := (others => (others => 'X'));
96      signal RAMbits2 : RAMtype := (others => (others => 'X'));
97      signal RAMbits3 : RAMtype := (others => (others => 'X'));

```

```

98
99 -- general read and write signals
100 signal RE : std_logic;
101 signal WE : std_logic;
102
103 -- data read from memory
104 signal MemData : std_logic_vector(31 downto 0);
105
106
107 begin
108
109
110 -- compute the general read and write signals (active low signals)
111 RE <= RE0 and RE1 and RE2 and RE3;
112 WE <= WE0 and WE1 and WE2 and WE3;
113
114
115 process
116     -- variable wr_data : std_logic_vector(31 downto 0);
117     begin
118
119         -- wait for an input to change
120         wait on RE, RE0, RE1, RE2, RE3, WE, WE0, WE1, WE2, WE3, MemAB;
121
122         -- first check if reading
123         if (RE = '0') then
124             -- reading, put the data out (check the address)
125             if ((to_integer(unsigned(MemAB)) >= START_ADDR0) and
126                 (to_integer(unsigned(MemAB)) - START_ADDR0 < (4 * RAM_SIZE))) then
127                 MemDB <= RAMbits0(to_integer(unsigned(MemAB(31 downto 2))) -
START_ADDR0 / 4);
128             elsif ((to_integer(unsigned(MemAB)) >= START_ADDR1) and
129                 (to_integer(unsigned(MemAB)) - START_ADDR1 < (4 * RAM_SIZE)))
then
130                 MemDB <= RAMbits1(to_integer(unsigned(MemAB(31 downto 2))) -
START_ADDR1 / 4);
131             elsif ((to_integer(unsigned(MemAB)) >= START_ADDR2) and
132                 (to_integer(unsigned(MemAB)) - START_ADDR2 < (4 * RAM_SIZE))) then
133                 MemDB <= RAMbits2(to_integer(unsigned(MemAB(31 downto 2))) -
START_ADDR2 / 4);
134             elsif ((to_integer(unsigned(MemAB)) >= START_ADDR3) and
135                 (to_integer(unsigned(MemAB)) - START_ADDR3 < (4 * RAM_SIZE))) then
136                 MemDB <= RAMbits3(to_integer(unsigned(MemAB(31 downto 2))) -
START_ADDR3 / 4);
137             else
138                 -- outside of any allowable address range - set output to X
139                 MemDB <= (others => 'X');
140             end if;
141
142             -- only set the bytes that are being read
143             if RE0 /= '0' then

```

```

144         MemDB(7 downto 0) <= (others => 'Z');
145     end if;
146     if RE1 /= '0' then
147         MemDB(15 downto 8) <= (others => 'Z');
148     end if;
149     if RE2 /= '0' then
150         MemDB(23 downto 16) <= (others => 'Z');
151     end if;
152     if RE3 /= '0' then
153         MemDB(31 downto 24) <= (others => 'Z');
154     end if;
155
156     else
157
158         -- not reading, send data bus to hi-Z
159         MemDB <= (others => 'Z');
160     end if;
161
162     -- now check if writing (falling edge: data and WE0-3 are valid)
163     if (WE'event and (WE = '0')) then
164         -- read current value at the target address
165         if ((to_integer(unsigned(MemAB)) >= START_ADDR0) and
166             (to_integer(unsigned(MemAB)) - START_ADDR0 < (4 * RAM_SIZE))) then
167             MemData <= RAMbits0(to_integer(unsigned(MemAB(31 downto 2))) -
START_ADDR0 / 4);
168         elsif ((to_integer(unsigned(MemAB)) >= START_ADDR1) and
169             (to_integer(unsigned(MemAB)) - START_ADDR1 < (4 * RAM_SIZE)))
then
170             MemData <= RAMbits1(to_integer(unsigned(MemAB(31 downto 2))) -
START_ADDR1 / 4);
171         elsif ((to_integer(unsigned(MemAB)) >= START_ADDR2) and
172             (to_integer(unsigned(MemAB)) - START_ADDR2 < (4 * RAM_SIZE)))
then
173             --wr_data := RAMbits2(to_integer(unsigned(MemAB(31 downto 2))) -
START_ADDR2 / 4);
174             MemData <= RAMbits2(to_integer(unsigned(MemAB(31 downto 2))) -
START_ADDR2 / 4);
175         elsif ((to_integer(unsigned(MemAB)) >= START_ADDR3) and
176             (to_integer(unsigned(MemAB)) - START_ADDR3 < (4 * RAM_SIZE)))
then
177             MemData <= RAMbits3(to_integer(unsigned(MemAB(31 downto 2))) -
START_ADDR3 / 4);
178         else
179             MemData <= (others => 'X');
180         end if;
181         wait for 0 ns; -- wait for data to be read
182
183         -- merge bytes being written (WE0-3 are still '0' on falling edge)
184         if WE0 = '0' then
185             -- wr_data(7 downto 0) := MemDB(7 downto 0);
186             MemData(7 downto 0) <= MemDB(7 downto 0);

```

```
end process;
```

```
231 | end behavioral;  
232 |
```